

Java OOP — Constructor Onwards

Constructors and the **this** keyword, with examples, comparisons, and practice.

1. What is a Constructor?

A **constructor** is a special member of a class that is automatically called when an object is created. It is mainly used to initialize an object's data.

```
Student s1 = new Student();

new Student() automatically calls
the Student constructor.
```

2. Constructor Rules

Rule 1: Constructor name must be the same as the class name.

Rule 2: A constructor has no return type, not even **void**.

Rule 3: It is automatically called when an object is created.

```
class Student {
    Student() {
        // constructor
    }
}
```

3. Simple Constructor

```
import java.util.*;

class Student {
    String name;
    int marks;

    Student() {
        System.out.println("Constructor called");
    }
}

public class Example {
    public static void main(String[] args) {
        Student s1 = new Student();
    }
}
```

Output: Constructor called

4. Parameterized Constructor

A parameterized constructor accepts values when an object is created, allowing the object to be initialized directly.

```
class Student {
    String name;
    int marks;

    Student(String n, int m) {
        name = n;
        marks = m;
    }
}
```

```
Student s1 = new Student("Sanjay", 75);
```

5. Complete Constructor Example

```
import java.util.*;

class Student {
    String name;
    int marks;

    Student(String name, int marks) {
        this.name = name;
        this.marks = marks;
    }

    void checkResult() {
        if (marks >= 40) {
            System.out.println(name + " Pass");
        } else {
            System.out.println(name + " Fail");
        }
    }
}

public class Example {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String name = sc.nextLine();
        int marks = sc.nextInt();

        Student s1 = new Student(name, marks);
        s1.checkResult();
    }
}
```

6. Default Constructor

If you do not write any constructor, Java provides a default no-argument constructor. Once you define your own constructor, Java does not automatically add a no-argument constructor.

```
class Student {
    String name;
    int marks;
}

Student s1 = new Student(); // works

class Employee {
    Employee(String name) {
    }
}

// Employee e = new Employee(); // error
// unless Employee() is also defined
```

7. Constructor vs Method

Constructor	Method
Same name as class	Can have any valid name
No return type	Has a return type or void

Automatically called during object creation	Normally called explicitly
Used mainly for initialization	Used to perform an operation
Example: Student()	Example: display()

8. What is the this Keyword?

this refers to the current object. It is especially useful when constructor parameters have the same names as instance variables.

```
class Student {
    String name;
    int marks;

    Student(String name, int marks) {
        this.name = name;
        this.marks = marks;
    }
}
```

9. Why Do We Need this?

When a parameter and an instance variable have the same name, **this** clearly identifies the current object's field.

```
this.name = name;
this.marks = marks;
```

```
this.name    → instance variable
name         → constructor parameter
```

```
this.marks   → instance variable
marks        → constructor parameter
```

10. Complete Employee Example

```
import java.util.*;

class Employee {
    String name;
    double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Salary: " + salary);
    }
}

public class Constructor2 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String name = sc.nextLine();
        double salary = sc.nextDouble();

        Employee e1 = new Employee(name, salary);
        e1.display();
    }
}
```

Example input: Sanjay, 50000

Output: Name: Sanjay Salary: 50000.0

11. this with Multiple Objects

```

Student s1 = new Student("Sanjay", 75);
Student s2 = new Student("Rahul", 90);

For s1:
this → s1

For s2:
this → s2

```

The **this** reference always represents the object whose constructor or instance method is currently executing.

12. this Inside a Method

```

class Student {
    String name;

    void display() {
        System.out.println(this.name);
    }
}

```

Inside an instance method, **this.name** means the name belonging to the current object.

13. Another Use of this

```

class Student {
    void display() {
        System.out.println("Hello");
    }

    void show() {
        this.display();
    }
}

```

Here, **this.display()** calls the display() method of the current object.

14. Logic-Building Approach

```

Identify the entity
↓
Identify its properties
↓
Create instance variables
↓
Identify initial values
↓
Create a constructor
↓
Use this.field = parameter
↓
Create the object
↓
Call methods

```

15. Practice Problem

Create a class **Employee** with **name**, **salary**, and **department**. Create a constructor Employee(String name, double salary, String department). Use **this** to initialize all three variables. Create display() to print the details. Use Scanner and create the Employee object in main.

```

Employee e1 =
    new Employee(name, salary, department);

```

16. Key Takeaways

- Constructor initializes an object.
- Constructor name must match the class name.
- Constructor has no return type.
- Parameterized constructors accept initial values.
- **this** refers to the current object.
- **this.name = name** assigns the parameter to the current object's field.
- Java class names are conventionally written in PascalCase.

Next OOP Topic: Constructor Overloading.